Objetivos formativos

Objetivos generales.

Que el alumno sea capaz de:

- Comprender el funcionamiento y saber diseñar los circuitos digitales que se utilizan en la implementación de un computador RISC sencillo (con 25 instrucciones de lenguaje máquina).
- Comprender el lenguaje máquina/ensamblador (SISA) de un computador RISC sencillo, la estructura interna del computador a nivel de circuitos digitales (en sus versiones SISC Harvard uniciclo, Harvard multiciclo y Von Neumann) y cómo este computador ejecuta programas en lenguaje máquina.

Objetivos específicos.

En este documento se detallan los objetivos específicos de la asignatura. Está formado por una lista de unos 100 objetivos. Es lo que el estudiante de IC debe saber o ser capaz de hacer al finalizar el curso. Estos objetivos, que sirven casi directamente para hacer una pregunta de examen, son los que evaluaremos en las pruebas de evaluación continua y en los exámenes de la asignatura. Esto es un contrato entre los profesores y los alumnos de IC. Nunca preguntaremos nada que no esté incluido en esta lista de objetivos específicos. Pero también forma parte del contrato que preguntaremos lo que esté aquí, aunque el profesor no lo haya explicado explícitamente en la clase. No todo lo que "entra para examen" lo explicará el profesor. Muchas de las cosas las tendréis que leer directamente de la documentación que os facilitamos y tendréis que hacer los ejercicios (trabajos para casa) que os proponemos. La lista de objetivos es útil para que os evaluéis vosotros mismos.

Los objetivos se han estructurado en 14 temas. Después del título de cada tema se formula uno o dos objetivos generales del tema y seguidamente se detalla la lista de objetivos específicos del tema. Cada objetivo específico se ha numerado, para podernos referir a él cuando os proponemos un ejercicio para hacer en casa o una pregunta en una prueba final de tema, por ejemplo.

De cara a la evaluación, los objetivos específicos se han clasificado en dos niveles de competencia: nivel B (objetivos o ejercicios **B**ásicos) y nivel A (**A**vanzados). En IC hay unos 100 objetivos, de los que 90 son de nivel B, como es normal en una asignatura de primeros cursos, y solo 10 son de nivel A. El nivel de competencia B se puede descomponer en dos subniveles: B1 (conocimiento, unos 50 objetivos) y B2 (comprensión, unos 40). Los niveles son jerárquicos en el sentido de que para conseguir un objetivo de nivel A hay que haber conseguido previamente algunos de nivel B2 y para conseguir estos algunos de nivel B1. Para responder a una pregunta que evalúa un objetivo de nivel B1 se requiere memorizar definiciones, símbolos, esquemas lógicos de dispositivos, etc. La respuesta se puede saber "de memoria". La resolución de un ejercicio para evaluar un objetivo de nivel B2 requiere aplicar una fórmula o un procedimiento conocido. La solución de estos ejercicios suele ser única. Sin embargo, la resolución de un problema de nivel A requiere tomar decisiones, elegir entre varios procedimientos, generar una idea nueva, analizarla, evaluarla y, si no sirve para solucionar el problema, modificarla y volver a analizar, etc. Estos problemas suelen admitir varias soluciones válidas.

El nivel de competencia B1, B2 o A, se muestra en el margen derecho del enunciado de cada objetivo específico.

> Al formular un objetivo específico se intenta acotar la complejidad del ejercicio que se puede poner para evaluar el objetivo, por lo que se indica el número y/o el tamaño máximo de algunos elementos que aparecen en el enunciado. Este número/tamaño da una idea aproximada de la complejidad del ejercicio. Por ejemplo, cuando se habla de programas de como máximo 10 instrucciones, se entiende que si es un código sencillo puede tener alguna instrucción más, si esto no supone un aumento sustancial de la complejidad del problema.

Notación y términos utilizados:

1. En algunos objetivos se usa la siguiente notación. Dado el vector X de n bits

   $$X = x_{n-1}x_{n-2} \ \ldots \ x_2x_1x_0 \qquad x_i \in \{0,1\}$$

   el valor numérico que representa el vector de bits si se interpreta como un número natural (*unsigned integer*) se denota como X_u, siendo

   $$X_u = \sum_{i=0}^{n-1} x_i 2^i$$

   mientras que el valor interpretando el vector de bits como un número entero (*signed integer*) se denota como X_s, siendo

   $$X_s = -x_{n-1}2^{n-1} + \sum_{i=0}^{n-2} x_i 2^i$$

2. Siempre que no se indique lo contrario, la salida de todos los **bloques** combinacionales y secuenciales son de n bits, los mismos que la o las palabras de entrada. Por ejemplo, para X e Y de n bits, la salida del bloque ADD(X,Y) es de n bits y esto no se indica explícitamente.
3. Usamos el término **circuito combinacional pequeño** para referirnos a un circuito combinacional con 4 señales (variables lógicas o bits) de entrada, como máximo.
4. Usamos el término **circuito secuencial**, o simplemente secuencial para referirnos a un circuito lógico secuencial síncrono. No se consideran los circuitos lógicos secuenciales asíncronos.
5. En todos los casos excepto en los objetivos 6.6 y 6.8 en los que se especifica que el circuito secuencial puede ser de Mealy o de Moore, cuando se habla de circuito secuencial se trata de un circuito secuencial de **Moore** (las salidas en un ciclo son función únicamente del estado, y no de las entradas, en ese mismo ciclo). Por lo tanto, en todos los objetivos que se habla de un **grafo de estados** nos referimos a un grafo de estados de Moore (el valor de las salidas se especifica en cada nodo).
6. Usamos el término **circuito secuencial pequeño** para referirnos a un circuito secuencial en el que el número de señales (variables lógicas o bits) de entrada más el logaritmo en base 2 del número de estados del circuito vale como máximo 4 (1 entrada y 8 estados, 2 entradas y 4 estados, o 3 entradas y 2 estados).

Notación y términos utilizados (se referencia como [#] al final del objetivos que lo requiere):

7. En algunos objetivos de los temas 7 y 8 se parte de un **grafo de estados incompleto** de la unidad de control de un procesador de propósito específico y se pide que se complete el grafo para que el sistema realice una funcionalidad descrita. En todos los casos se da la información necesaria del grafo para que sólo haya una única solución correcta o muy pocas variantes (son objetivos de nivel B2). En el grafo que se da pueden faltar nodos, arcos, etiquetas de arcos y/o la especificación de las salidas de algunos nodos. Cuando se trata de la unidad de control de un procesador de propósito específico con la UPG, UPG+IO o UPG+IO+Mem, las salidas se indican mediante mnemotécnicos o mediante el vector de bits correspondiente de la palabra de control (que puede ser expresado en hexadecimal).
8. Todos los grafos de estados que aparecen en los objetivos (que se dan o que hay que dibujar o completar) tiene a lo sumo **10 estados**.
9. En algunos objetivos de los temas 6, 7 y 8 se parte de lo que denominamos una **descripción funcional de un bloque/circuito secuencial o de un procesador de propósito específico**. Esta descripción relaciona, de manera más o menos formal, el valor de las salidas en un ciclo en función de las entradas en ciclos anteriores mediante un texto explicativo y/o expresiones algebraicas y/o un fragmento de código en un pseudolenguaje de alto nivel. La descripción puede completarse con dibujos explicativos del sistema y/o cronogramas simplificados de las salidas para secuencias concretas de las entradas.
10. En algunos objetivos de los temas 6 en adelante se habla de **cronograma simplificado** de un circuito secuencial o procesador de propósito específico. Se trata de un cronograma en el que se expresa el valor de cada señal desde el principio de cada ciclo el valor que tendrá la señal en ese ciclo una vez estabilizada. Es un cronograma considerando nulos todos los tiempos de propagación. Se suele representar en forma de tabla, con una casilla por señal y ciclo en la que se escribe el valor.
11. Todos los **esquemas lógicos a bloques** de la unidad de proceso de un procesador de propósito específico que aparecen en los objetivos del tema 7 (de los que se parte o que hay que dibujar o completar) no tienen más de 8 elementos entre puertas lógicas, biestables, bloques combinacionales enunciados en los objetivos de los temas 3, 4 y 5, multiplexores de buses y registros REG y REGwLd.
12. En todos los objetivos que haya que **completar un esquema a bloques** de un circuito combinacional o secuencial se dará la información y/o las restricciones necesarias para que sólo haya una única solución correcta (o muy pocas variantes) ya que estos son objetivos de nivel B2.

1. Introducción a los computadores

Definir conceptos básicos sobre computadores.

1.1. Definir con palabras propias los siguientes conceptos: computador, circuito digital (lógico), información digital, dígito, bit, codificación de la información, programa, algoritmo, lenguaje de programación, lenguaje de alto nivel, lenguaje ensamblador, lenguaje máquina, subsistemas: procesador, memoria, entrada/salida y buses. B1

2. Representación de números naturales

Comprender el sistema de numeración convencional en base b para la representación de números naturales y en particular el caso binario (b=2) y saber cambiar la representación de un número entre bases 2, 10 y 16.

2.1. Escribir la fórmula que da el valor de un número natural en función de los n dígitos que lo representan en el sistema convencional en base b. B1

2.2. Expresar el rango de los números naturales que se pueden representar en el sistema de numeración convencional en base b, en función de la base b y del número de dígitos n. B1

2.3. Representar con n+1 dígitos un número natural representado inicialmente en base 2 (binario), base 16 (hexadecimal) o base 10 (decimal) con n dígitos (algoritmo de extensión de rango). B1

2.4. Cambiar la representación de un número natural de base 2 (binario), de base 16 (hexadecimal) o de base 10 (decimal) a otra de esas bases. B2

3. Circuitos lógicos combinacionales

Comprender qué es un circuito lógico combinacional y ser capaz de analizar circuitos (obtener la tabla de verdad y los tiempos de propagación) y de sintetizarlos (mediante uno de los siguientes conjuntos de dispositivos: puertas Not, And y Or (en suma de minterms), un decodificador y puertas Or o una memoria ROM).

Introducción

3.1. Definir con palabras propias los siguientes conceptos: circuito lógico combinacional, variable lógica binaria, función lógica y tabla de verdad. B1

3.2. Especificar circuitos combinacionales pequeños. Obtener la tabla de verdad de un circuito combinacional pequeño a partir de un texto que describe su funcionamiento. B2

3.3. Dibujar el símbolo, especificar la tabla de verdad y expresar la operación lógica que implementa cada una de las puertas Not, And y Or (las puertas And y Or con dos o más entradas). B1

3.4. A partir del esquema lógico de un circuito formado por puertas lógicas y otros dispositivos combinacionales interconectados entre sí, indicar si es un esquema válido para un circuito combinacional, y en caso de que no lo sea indicar la razón. B2

Análisis

3.5. Obtener la tabla de verdad de un circuito combinacional pequeño a partir de su esquema lógico. El circuito está formado en la interconexión de no más de 8 puertas (Not, And, Or). El esquema puede contener también puertas Xor y uno o dos símbolos de dispositivos combinacionales con no más de tres entradas y tres salidas de los que se dé su tabla de verdad. B2

Síntesis

3.6. A partir de la tabla de verdad de un circuito combinacional pequeño: B2

3.6.1. escribir la expresión lógica en suma de minterms de cada salida y

3.6.2. dibujar el esquema lógico del circuito como implementación directa de la expresión en suma de minterms de cada salida.

3.7. Dibujar el símbolo, especificar la tabla de verdad, dar una expresión lógica de cada una de las salidas, dibujar el esquema lógico interno usando puertas Not, And y Or y describir con palabras propias la funcionalidad de cada uno de los siguientes bloques combinacionales: B1

3.7.1. puerta Xor de dos entradas (Xor-2),

3.7.2. decodificador 1-2 (Dec-2-1), 2-4 (Dec-2-4), etc. con y sin entrada de habilitación,

3.7.3. multiplexor 2-1 (Mx-2-1).

3.8. A partir de la tabla de verdad de un circuito combinacional pequeño, dibujar un esquema lógico que lo implemente mediante un decodificador y puertas Or. B2

3.9. Describir con palabras propias la funcionalidad de una memoria ROM. Dibujar el esquema lógico interno que modela de forma sencilla una memoria ROM mediante un decodificador, una matriz de interconexión y puertas Or. B1

3.10. Especificar el tamaño mínimo de la ROM para sintetizar un circuito combinacional con n entradas y m salidas (indicando: número_de_palabras x bits_por_palabra). B1

3.11. A partir de la tabla de verdad de un circuito combinacional pequeño, dibujar el esquema lógico que lo implementa con una sola ROM (interconectar las entradas y salidas de la ROM con las entradas y salidas del circuito y especificar el contenido de la ROM). B2

Tiempos de propagación

3.12. Dibujar el cronograma de las señales que se piden de un circuito combinacional pequeño a partir de su esquema lógico formado por puertas Not, And y Or, el retardo de cada puerta y la evolución temporal de las señales de entrada. B2

3.13. Indicar el camino crítico (o uno de ellos, si hay varios) y los tiempos de propagación desde cada entrada a cada salida de un circuito combinacional a partir de su esquema lógico, conocidos los tiempos de propagación desde cada entrada a cada salida de todos los elementos del circuito. El esquema lógico consiste en la interconexión de no más de 8 de los siguientes elementos: puertas Not, And, Or y Xor, bloques combinacionales decodificador 1-2 y 2-4, multiplexor 2-1, memoria ROM de no más de 4 bits de dirección y/o otros bloques combinacionales con no más de tres entradas y tres salidas. B2

Álgebra de Boole. Circuitos equivalentes y minimización usando mapas de Karnaugh

3.14. Listar los axiomas que definen el álgebra de Boole. B1

3.15. A partir de los axiomas demostrar propiedades/teoremas del álgebra de Boole. A

3.16. A partir de la tabla de verdad de un circuito combinacional pequeño y de una lista de axiomas y propiedades/teoremas, dibujar varios esquemas lógicos equivalentes, usando puertas Not, And y Or. Cada esquema debe ser una implementación directa de las expresiones lógicas obtenidas mediante la manipulación de la expresión en suma de minterms de cada una de las salidas. Elegir los esquemas que requieren menor hardware y/o menor tiempo de propagación. A

3.17. A partir de la tabla de verdad de un circuito combinacional pequeño, o de la expresión en suma de minterms de cada una de sus salidas, obtener la expresión mínima en producto de sumas mediante un mapa de Karnaugh. B2

4. Bloques aritméticos combinacionales para naturales

Realizar las operaciones aritméticas básicas sobre números naturales representados en binario.

Comprender el funcionamiento y diseñar bloques combinacionales (circuitos combinacionales que manipulan palabras de n bits (para $n \leq 16$) para la realización de operaciones aritméticas con números naturales representados en binario.

Bloques aritméticos básicos

4.1. Aplicar los algoritmos aritméticos de las operaciones básicas sobre números naturales representados en binario con n bits ($n \leq 16$). En concreto, mediante la secuencia adecuada de operaciones lógicas con los bits que representan a los operandos, obtener el vector de bits que representa el resultado de las siguientes operaciones aritméticas, incluida la detección de resultado no representable en n bits: B2

- 4.1.1. suma,
- 4.1.2. resta,
- 4.1.3. multiplicación por potencias de dos,
- 4.1.4. división por potencias de dos.

4.2. Dibujar el símbolo, especificar la tabla de verdad, dar una expresión lógica de cada una de las salidas, dibujar el esquema lógico interno usando puertas Not, And y Or y describir con palabras propias la funcionalidad de cada uno de estos bloques combinacionales: B1

- 4.2.1. Half-adder (Ha),
- 4.2.2. Full-adder (Fa) (también usando dos Half-adders y una Or) y
- 4.2.3. Full-subtractor (Fs).

4.3. Dibujar el símbolo y el esquema lógico interno de cada uno de los siguientes bloques combinacionales que realizan operaciones aritméticas sobre números naturales (X e Y son vectores de n bits, para $n \leq 16$, que representan en binario a los números naturales X_u e Y_u respectivamente). B1

- 4.3.1. Incrementador binario, INC(X), que calcula $X_u + 1$ cuando este valor es representable en binario con n bits, a partir de Half-adders (Ha). (Con señal de resultado no representable en n bits).
- 4.3.2. Sumador binario, ADD(X,Y), que calcula $X_u + Y_u$ cuando este valor es representable en binario con n bits, a partir de Full-adders (Fa). (Con señal de resultado no representable en n bits, c, *carry*.)
- 4.3.3. Restador binario, SUB(X,Y), que calcula $X_u - Y_u$ cuando este valor es representable en binario con n bits, a partir de Full-subtractors (Fs). (Con señal de resultado no representable en n bits, b, *borrow*).

4.4. Dibujar el símbolo y el esquema lógico interno de cada uno de los siguientes bloques combinacionales que realizan desplazamientos lógicos. (En la interpretación aritmética de los desplazamientos como multiplicaciones y divisiones por potencias de dos, X es un vector de n bits, para $n \leq 16$, que representa en binario al número natural X_u). B1

- 4.4.1. Desplazador a la izquierda de k bits, SL-k(X), para $k \leq 16$, que calcula $X_u \times 2^k$ cuando este valor es representable en binario con n bits. (Con señal de resultado no representable en n bits).
- 4.4.2. Desplazador lógico a la derecha de k bits, SRL-k(X), para $k \leq 16$, que calcula $\frac{X_u}{2^k}$.

4.5. Dibujar el símbolo y el esquema lógico interno usando puertas del bloque Z(X), *Zero*, que compara X con cero. X es un vector de n bits, para $n \leq 16$. La salida del bloque, z, es de un bit y vale 1 cuando $X_u = 0$ y vale 0 en cualquier otro caso. B1

4.6. Dibujar el esquema lógico interno de cada uno de los siguientes bloques combinacionales que comparan dos números naturales (X e Y son vectores de n bits, para $n \leq 16$, que representan en binario a los números naturales X_u e Y_u respectivamente) a partir de un restador binario de n bits con salida de *borrow*, b, y puertas lógicas. La salida del bloque es de un bit, que vale 1 cuando el resultado de la comparación es cierto y 0 cuando es falso. B1

4.6.1. EQ(X,Y), *Equal*, Igual, X = Y.

4.6.2. LTU(X,Y), *Less Than (Unsigned)*, Menor que (para naturales), $X_u < Y_u$.

4.6.3. LEU(X,Y), *Less than or Equal (Unsigned)*, Menor o igual (para naturales), $X_u \leq Y_u$.

Bloques no aritméticos

4.7. Dibujar el símbolo y el esquema lógico interno de cada uno de los siguientes bloques combinacionales que realizan operaciones lógicas bit a bit (bit wise), AND(X,Y), OR(X,Y), XOR(X,Y) y NOT(X), a partir de puertas lógicas And, Or, Xor y Not respectivamente. (X e Y son vectores de n bits, para $n \leq 16$). B1

4.8. Dibujar el símbolo y el esquema lógico interno de los multiplexores 4-1 (Mx-4-1) y 8-1 (Mx-8-1) a partir de multiplexores 2-1 (Mx-2-1). B1

Análisis de circuitos combinacionales a bloques

4.9. Dado el esquema lógico a bloques de un circuito combinacional, formado por no más de 4 de entre los bloques INC, ADD, SUB, SL-k, SRL-k, Z, EQ, LTU, LEU, AND, OR, XOR, NOT, MUX, Mx y puertas lógicas, escribir el valor de las señales y buses de salida una vez estabilizados, a partir del valor de las señales y buses de entrada. Los bloques pueden tener, además, salida de resultado no representable en n bits para números naturales, cuando esto proceda. B2

Diseño de nuevos bloques

4.10. Completar el diseño de nuevos bloques combinacionales. A partir de un texto, una expresión algebraica o de un fragmento de código en un pseudolenguaje de alto nivel que describe la funcionalidad de un bloque combinacional para realizar operaciones sobre vectores de n bits (típicamente operaciones aritméticas sobre números naturales) y del esquema interno (usando bloques más sencillos) incompleto, completarlo dibujando los bloques y las interconexiones que faltan para que implemente la funcionalidad descrita. El circuito completo no tendrá más de 8 elementos entre puertas lógicas y los bloques combinacionales enunciados en los objetivos del 4.2 al 4.8. [12] B2

4.11. Diseñar nuevos bloques combinacionales. A partir de un texto o una expresión algebraica que describe la funcionalidad de un bloque combinacional que realiza operaciones sobre vectores de n bits (típicamente operaciones aritméticas sobre números naturales), dibujar un esquema a bloques que lo implemente (diseño ad-hoc). El circuito completo no tendrá más de 8 elementos entre puertas lógicas y los bloques combinacionales enunciados en los objetivos del 4.2 al 4.8. [11] A

5. Números enteros: representación, operaciones básicas y bloques aritméticos combinacionales

Comprender el sistema de representación de números enteros "complemento a dos", aplicar los algoritmos aritméticos para realizar las operaciones aritméticas básicas sobre vectores de bits que representan números enteros en complemento a dos y dibujar los esquemas lógicos internos de los bloques combinacionales que implementan dichos algoritmos.

5.1. Justificar las ventajas de la representación complemento a dos frente a signo y magnitud. B1

5.2. Escribir la fórmula que da el valor de un número entero en función de los n bits que lo representan en complemento a dos. B1

5.3. Expresar el rango de los números enteros que se pueden representar en complemento a dos en función del número de bits. B1

5.4. Dado un número entero representado en complemento a dos con n bits, obtener su representación en complemento a dos con n+1 bits (algoritmo de extensión de rango). B2

5.5. Dibujar el símbolo y el esquema lógico interno usando solamente cables del bloque k-SE(X), *Sign Extension*, que implementa el algoritmo de extensión de rango para números enteros en complemento a dos. X es un vector de n bits, para $n \leq 16$. Los n bits de la salida del bloque representan al mismo número entero que los *k* bits de menor peso de la entrada ($k < n$): k-SE(X)s = X<k-1..0>s. B1

5.6. Obtener la representación en complemento a dos de un número entero a partir de su representación en signo y magnitud decimal y viceversa. B2

5.7. Aplicar algoritmos aritméticos de operaciones básicas sobre números enteros representados en complemento a dos con n bits ($n \leq 16$). En concreto, mediante la secuencia adecuada de operaciones con los bits que representan a los operandos, obtener el vector de bits que representa el resultado de las siguientes operaciones aritméticas, incluida la detección de resultado no representable en n bits: B2

5.7.1. suma,

5.7.2. cambio de signo,

5.7.3. resta (mediante cambio de signo y suma),

5.7.4. multiplicación por potencias de dos y

5.7.5. división por potencias de dos.

5.8. Dibujar el esquema lógico interno de cada uno de los siguientes bloques combinacionales que realizan operaciones aritméticas con números enteros (X e Y son vectores de n bits, para $n \leq 16$, que representan en complemento a dos a los números enteros X_s e Y_s respectivamente). B1

5.8.1. Sumador binario, ADD(X,Y), que calcula $X_s + Y_s$ cuando este valor es representable en complemento a dos con n bits, a partir de Full-adders (Fa). (Con señal de resultado no representable en n bits, v, *overflow*).

5.8.2. Negador, CS(X), *change of sign*, que calcula $-X_s$ cuando este valor es representable en binario con n bits, a partir de Half-adders (Ha) y puertas Not. (Con señal de resultado no representable en n bits).

5.8.3. Restador binario, SUB(X,Y), que calcula $X_s - Y_s$ cuando este valor es representable en complemento a dos con n bits, a partir de Full-adders (Fa) y puertas Not. (Con señal binaria de resultado no representable en n bits, v, *overflow*).

5.9. Dibujar el símbolo y el esquema lógico interno del bloque combinacional SRA-k, que realiza desplazamientos aritméticos a la derecha, con sólo cables. (En la interpretación aritmética de B1

los desplazamientos a la derecha como divisiones por potencias de dos, X es un vector de n bits, para $n \leq 16$, que representa en binario al número entero X_S. El bloque SRA-k(X) calcula $\frac{X_S}{2^k}$ para $k \leq 16$).

5.10. Dibujar el símbolo y el esquema lógico interno de cada uno de los siguientes bloques combinacionales que comparan dos números enteros (X e Y son vectores de n bits, para $n \leq 16$, que representan en complemento a dos a los números enteros X_S e Y_S respectivamente) a partir de un restador binario de n bits con salida de *overflow*, v, y puertas lógicas. La salida del bloque es de un bit, que vale 1 cuando el resultado de la comparación es cierto y 0 cuando es falso. B1

5.10.1. LT(X,Y), *Less Than (Signed)*, Menor que (para enteros), $X_S < Y_S$.

5.10.2. LE(X,Y), *Less than or Equal (Signed)*, Menor o igual (para enteros), $X_S \leq Y_S$.

Análisis de circuitos combinacionales a bloques

5.11. Dado el esquema lógico a bloques de un circuito combinacional, formado por no más de 4 de entre los bloques del objetivo 4.9 y los bloques SE, CS, SRA-k, LT y LE y puertas lógicas, escribir el valor de las señales y buses de salida una vez estabilizados, a partir del valor de las señales y buses de entrada. Además, los bloques pueden tener salida de resultado no representable en n bits tanto para números naturales como para enteros. B2

Diseño de nuevos bloques

5.12. Completar el diseño de nuevos bloques combinacionales. A partir de un texto, una expresión algebraica o de un fragmento de código en un pseudolenguaje de alto nivel que describe la funcionalidad de un bloque combinacional para realizar operaciones sobre vectores de n bits (típicamente operaciones aritméticas sobre números naturales o enteros) y del esquema interno (usando bloques más sencillos) incompleto, completarlo dibujando los bloques y las interconexiones que faltan para que implemente la funcionalidad descrita. El circuito completo no tendrá más de 8 elementos entre puertas lógicas, bloques combinacionales aritméticos para naturales o enteros y multiplexores. [12] B2

5.13. Diseñar nuevos bloques combinacionales. A partir de un texto o una expresión algebraica que describe la funcionalidad de un bloque combinacional que realiza operaciones sobre vectores de n bits (típicamente operaciones aritméticas sobre números naturales o enteros), dibujar un esquema a bloques que lo implemente (diseño ad-hoc). El circuito completo no tendrá más de 8 elementos entre puertas lógicas, bloques combinacionales aritméticos para naturales o enteros y multiplexores. [11] A

6. Circuitos lógicos secuenciales

Comprender qué es un circuito lógico secuencial (caso general de Mealy y caso particular de Moore) y ser capaz de analizar y sintetizar (con el número mínimo de biestables D activados por flanco) circuitos secuenciales de Moore. Calcular el tiempo de ciclo mínimo del circuito.

Introducción

6.1. Definir circuito lógico secuencial. Expresar con palabras propias qué es un circuito secuencial y las diferencias entre combinacional y secuencial. B1

6.2. Enunciar ejemplos de funcionalidad de circuitos lógicos que necesariamente tengan que ser secuenciales (que no puedan ser combinacionales porque necesiten memoria y sincronización). B1

6.3. Definir biestable D activado por flanco (sin y con entradas asíncronas de Set, S, y Reset., R). Dibujar el símbolo usado en los esquemas lógicos para representarlo. Expresar con palabras propias su funcionalidad. Expresar su funcionalidad mediante las tablas de transiciones (tabla de verdad del estado siguiente) y de salida y/o su grafo de estados. B1

6.4. Dibujar el cronograma de la señal de salida Q de un biestable D activado por flanco a partir del cronograma de la señal de entrada D y de la señal de reloj (y de las señales S y R en caso de tenerlas). B2

6.5. Definir el bloque secuencial REG, registro de n bits. Dibujar su símbolo usado en los esquemas lógicos. Dibujar su esquema lógico interno a partir de n biestables D activados por flanco. B1

6.6. A partir del esquema lógico de un circuito indicar si es un esquema válido para un circuito secuencial, y en caso de que no lo sea indicar la razón. Si es válido indicar si es un secuencial de Moore o de Mealy. El circuito está formado por la interconexión de no más de 10 elementos entre puertas, bloques combinacionales y biestables D activados por flanco. B2

6.7. Expresar con palabras propias qué condición tiene que cumplir el tiempo de ciclo (el periodo) de la señal de reloj de un circuito formado por una red de dispositivos combinacionales y biestables D interconectados entre si formando un circuito secuencial válido. B1

6.8. Dibujar el esquema general de un circuito secuencial mediante los bloques: combinacional de estado siguiente, registro de estado y combinacional de salida, para los casos de Mealy y de Moore con n señales de entrada, m *de* salida y k biestables. B1

6.9. Cambiar la forma de especificar la funcionalidad de un circuito secuencial pequeño. A partir las tablas de verdad del estado siguiente (tabla de transiciones) y de las salidas, dibujar el grafo de estados, y viceversa. B2

6.10. A partir del grafo de estados de un circuito secuencial pequeño y de una secuencia concreta de valores de las entradas para un determinado número de ciclos, completar el cronograma simplificado que muestra el comportamiento del circuito (valor del estado y las salidas) para cada uno de esos ciclos. [10] B2

Síntesis

6.11. Completar el grafo de estados de un circuito secuencial pequeño a partir de un grafo incompleto y de una descripción funcional del circuito. [9] B2

6.12. Especificar circuitos secuenciales pequeños. A partir de una descripción funcional de un circuito secuencial pequeño, dibujar la caja (indicando las señales de entrada y salida) que simboliza al circuito y dibujar su grafo de estados. [9] A

6.13. Sintetizar circuitos secuenciales pequeños. A partir del grafo de estados de un circuito secuencial pequeño, dibujar un esquema lógico que lo implemente con el mínimo número de biestables D y alguna de las técnicas de síntesis de circuitos combinacionales (con puertas Not, And y Or en suma de minterms, con decodificador y puertas Or o con una ROM). La técnica para sintetizar el circuito combinacional que calcula el estado siguiente puede ser distinta que la usada para el circuito que calcula las salidas. También se puede usar una sola ROM para calcular el estado siguiente y las salidas junto con un multiplexor de buses para seleccionar el estado siguiente en función de las entradas. B2

Análisis lógico y temporal

6.14. Analizar circuitos secuenciales pequeños. Dibujar el grafo de estados a partir del esquema lógico del circuito. El esquema lógico consiste en la interconexión de no más de 10 de los siguientes elementos: biestables D activados por flanco, puertas Not, And, Or y Xor y los bloques combinacionales decodificador 1-2 y 2-4, multiplexor 2-1 y 4-1 y memoria ROM de no más de 4 bits de dirección. B2

6.15. Especificar el camino crítico (o uno de ellos, si hay varios) y calcular el tiempo de ciclo mínimo de un circuito secuencial pequeño a partir de su esquema lógico, dados los tiempos de propagación de cada entrada a cada salida de los elementos del circuito. El esquema lógico consiste en la interconexión de no más de 10 de los siguientes elementos: biestables D activados por flanco, puertas Not, And, Or y Xor y/o los bloques combinacionales decodificador 1-2 y 2-4 y/o multiplexor 2-1 y 4-1 y/o memoria ROM de no más de 4 bits de dirección y/o otros bloques combinacionales con no más de tres entradas y tres salidas. B2

6.16. Dibujar el cronograma de las señales de salida del circuito secuencial pequeño a partir de su esquema lógico, dados los tiempos de propagación de los elementos del circuito, el estado inicial y el cronograma de las señales de entrada. El esquema lógico consiste en la interconexión de no más de 10 de los siguientes elementos: biestables D activados por flanco, puertas Not, And, Or y otros bloques combinacionales de los que nos deben dar su implementación interna con puertas o su comportamiento temporal. B2

7. Procesadores de propósito específico

Comprender el funcionamiento y saber diseñar procesadores de propósito específico que manipulan palabras de n bits (para $n \leq 16$). El procesador está formado por una unidad de proceso y una unidad de control. La unidad de proceso se construye mediante un diseño ad-hoc con bloques combinacionales y secuenciales y la unidad de control se especifica mediante un grafo de estados de Moore.

Bloques combinacionales y secuenciales

7.1. Dibujar el símbolo y el esquema lógico interno de cada uno de los siguientes bloques combinacionales. B1

7.1.1. Multiplexor de buses 2-1 (MUX-2-1), a partir de multiplexores 2-1 (Mx-2-1), para $n \leq 16$.

7.1.2. Multiplexores de buses 4-1 (MUX-4-1) y 8-1 (MUX-8-1) a partir de multiplexores de buses 2-1 (MUX-2-1), para $n \leq 16$.

7.2. Dibujar el símbolo y el esquema lógico interno del registro de n bits con control de carga, REGLd, a partir de un registro de n bits y un multiplexor de buses 2-1. B1

7.3. Completar el cronograma simplificado de las salidas de un bloque secuencial a partir de su descripción funcional, de su estado inicial y de una secuencia concreta de valores de las entradas. [9] [10] B2

7.4. Completar el diseño de nuevos bloques secuenciales. A partir de la descripción funcional de un bloque secuencial que procesa palabras de n bits y un esquema a bloques incompleto, completarlo dibujando los bloques y las interconexiones que faltan para que implemente la funcionalidad descrita. [9] [11] [12] B2

7.5. Diseñar nuevos bloques secuenciales. A partir de la descripción funcional de un bloque secuencial que procesa palabras de n bits dibujar un esquema a bloques que lo implemente (diseño ad-hoc). [9] [11] A

Unidad de proceso + Unidad de control

7.6. Completar el cronograma simplificado de las salidas de un procesador de propósito específico a partir de la descripción funcional del procesador o del esquema lógico a bloques de su unidad de proceso y del grafo de estados de su unidad de control, del estado inicial y de una secuencia concreta de valores de las entradas. [9] [10] [11] B2

7.7. Completar el diseño de un procesador de propósito específico. A partir de la descripción funcional de un procesador de propósito específico y de un esquema a bloques incompleto de su unidad de proceso y/o de un grafo de estados incompleto que especifica su unidad de B2

control, completar el diseño para que implemente la funcionalidad descrita. [7] [8] [9] [11] [12]

7.8. Diseñar procesadores de propósito específico. A partir de la descripción funcional de un procesador de propósito específico, dibujar el esquema a bloques de la unidad de proceso (diseño ad-hoc) y el grafo de estados de la unidad de control. [8] [9] [11] A

7.9. Indicar el camino crítico (o uno de ellos si hay varios) y calcular el tiempo de ciclo mínimo de un procesador de propósito específico formado por unidad de proceso (especificada mediante su esquema lógico con no más de 8 bloques digitales) y unidad de control (implementada con alguna de las técnicas del objetivo 6.13), dados todos los tiempos de propagación de los dispositivos usados en su implementación. B2

8. Unidad de proceso general

Conocer la estructura a nivel de bloques de la unidad de proceso general (UPG) y el circuito lógico interno de cada bloque que la forma (banco de registros y unidad aritmético lógica) y comprender su funcionamiento.

Diseñar la unidad de control de propósito específico (UCE), especificando su grafo de estados, para que junto con la unidad de proceso general (UPG) implementen una funcionalidad concreta donde las entradas y salidas de datos son síncronas.

Bloques combinacionales y secuenciales multifunción

8.1. Dibujar el símbolo y el esquema lógico interno de la unidad aritmético lógica (AL) que procesa palabras, X e Y, de n bits, $n \leq 16$, con la siguiente funcionalidad según el valor de la señal de control F de 3 bits (cuya codificación se dará), a partir de los bloques aritméticos y lógicos de n bits que implementan cada función y un multiplexor de buses 8-1: AND(X,Y), OR(X,Y), XOR(X,Y), NOT(X), ADD(X,Y), SUB(X,Y), SHA(X,Y) y SHL(X,Y). B1

8.2. Dibujar el símbolo y el esquema lógico interno de la unidad de comparación (CMP) que compara palabras, X e Y, de n bits, $n \leq 16$, generando una salida W de n bits que toma valor $W_u = 1$ cuando la relación es cierta y $W_u = 0$ cuando es falsa. Para la implementación se usa un restador de dos números de n bits con salidas de borrow y overflow, puertas lógicas y un multiplexor 8-1. Se evalúan las siguientes relaciones según la señal F de 3 bits (cuya codificación se dará): LT(X,Y), LE(X,Y), EQ(X,Y), LTU(X,Y) y LEU(X,Y). B1

8.3. Dibujar el símbolo y el esquema lógico interno de la unidad aritmético lógica usada para implementar la unidad de proceso general, ALUupg, que tiene dos buses de entrada X e Y y uno de salida W de 16 bits, así como dos buses de entrada para seleccionar la funcionalidad de la ALUupg, F de 3 bits y OP de dos bits (cuya codificación se dará). Tiene también una señal de salida, z, que vale 1 cuando W_u es igual a 0 y 0 en cualquier otro caso. Para implementar la ALUupg se usan los bloques AL, CMP, MUX-4-1 y Z. La funcionalidad es la conjunta de los bloques AL y CMP. B1

8.4. Dibujar el símbolo y el esquema lógico interno de un banco de 8 registros de n bits cada uno con dos buses de lectura y 1 bus de escritura, REGFILE, a partir de un decodificador 3-8, 8 registros de n bits con control de carga y dos multiplexores de buses 8-1. B1

8.5. Dado el esquema lógico a bloques de un circuito (que puede ser combinacional o secuencial), formado por no más de 4 de entre los bloques AL, CMP, ALUupg, REG, REGwLd, REGFILE y MUX: a) escribir el valor concreto de las señales y buses de salida una vez estabilizados a partir del valor de las señales y buses de entrada, si es un circuito combinacional o b) completar el cronograma simplificado de algunas señales y buses del sistema a partir de la secuencia de valores de las señales/buses de entrada y del estado inicial de los registros, si es un circuito secuencial. B2

Unidad de proceso general (UPG)

8.6. Dibujar el símbolo y el esquema lógico interno de la Unidad de Proceso General (UPG) usando los bloques combinacionales multiplexor de buses 2-1, MUX-2-1, y unidad aritmético lógica, ALUupg, y el bloque secuencial banco de registros, REGFILE. B1

8.7. Expresar con mnemotécnicos cada una de las acciones que se pueden realizar en un ciclo en la UPG a partir de un texto que describe cada una de las posibles acciones y viceversa. Indicar los casos en que la acción descrita no puede realizarse en la UPG en un ciclo y explicar el motivo. B1

8.8. Indicar el valor que debe tener cada uno de los bits de la palabra de control para que la UPG realice, durante un ciclo, una acción concreta especificada mediante mnemotécnicos. Expresar la palabra de control de forma compacta en hexadecimal. Indicar los casos en que la acción descrita no puede realizarse en la UPG en un ciclo y explicar el motivo. B2

8.9. Indicar el valor de los registros de la UPG después de ejecutar una acción durante un ciclo o una secuencia de acciones que se ejecutan en ciclos consecutivos. Cada acción se especifica con mnemotécnicos y si se trata de una secuencia de acciones que se ejecutan en varios ciclos consecutivos se dará el grafo de estados de la unidad de control usando mnemotécnicos para las palabras de control. [8] B2

8.10. Completar el cronograma simplificado con el valor de algunas señales/buses de la UPG a partir del grafo de estados de la unidad de control de propósito específico que la gobierna y del estado del procesador (estado en el que se encuentra la unidad de control y valor de los registros de la UPG en el primer ciclo en el que se debe completar el cronograma). [8] [10] B2

Diseño de procesadores de propósito específico con la UPG y entrada/salida síncrona

8.11. Completar el diseño de un procesador de propósito específico formado por la Unidad de Proceso General (UPG) y una unidad de control de propósito específico (UCE). La unidad de control puede tener varias señales de entrada: z que viene de la ALU de la UPG y otras de control de la entrada/salida de datos síncrona. A partir de la descripción funcional de un procesador de propósito específico y de un grafo de estados incompleto que especifica su unidad de control, completar el grafo para que realice la funcionalidad descrita. [7] [8] [9] B2

8.12. Diseñar un procesador de propósito específico formado por la unidad de proceso general (UPG) y una unidad de control de propósito específico (UCE). La unidad de control puede tener varias señales de entrada: z que viene de la ALU de la UPG y otras de control de la entrada/salida de datos síncrona. A partir de la descripción funcional de un procesador de propósito específico, dibujar el grafo de estados de la unidad de control para realizar la funcionalidad descrita. [8] [9] A

9. Entrada/salida

Comprender el protocolo de comunicación asíncrona de cuatro pasos (*Four-Cycle Handshaking*).

Conocer y comprender el subsistema de entrada/salida sencillo (IO_{sim}) donde las entradas y salidas de datos pueden ser asíncronas utilizando el protocolo de hadshaking de cuatro pasos sobre puertos de entrada/salida (con direcciones simbólicas REQ, ACK y DATA-IN o DATA-OUT).

Diseñar la unidad de control de propósito específico (UCE), especificando su grafo de estados, para que junto con la unidad de proceso general (UPG) y el subsistema sencillo de entrada/salida (IOsim) implementen una funcionalidad concreta donde las entradas y salidas de datos son asíncronas y utilizan el protocolo de hadshaking de cuatro pasos sobre puertos de entrada/salida (con direcciones simbólicas REQ, ACK y DATA-IN o DATA-OUT).

Conocer y comprender el subsistema de entrada/salida basado en un teclado y una impresora sencillos ($IO_{KeyPrint}$) donde la entrada y salida de datos se realiza mediante encuesta sobre el registro de estado del controlador y efecto lateral (puesta a cero del registro de estado) al acceder al registro de datos.

Diseñar la unidad de control de propósito específico (UCE), especificando su grafo de estados, para que junto con la unidad de proceso general (UPG) y el subsistema de entrada/salida con teclado e impresora sencillos ($IO_{KeyPrint}$) implementen una funcionalidad concreta.

Subsistema sencillo de entrada/salida. Entrada/salida asíncrona

9.1. Dibujar el esquema lógico del subsistema sencillo de entrada/salida (IO_{sim}: SIMPLE-INPUT-OUTPUT) que se conecta a la UPG y que está formado por tres registros de entrada cuyo contenido puede cargar el usuario mediante teclados hexadecimales y tres registros de salida cuyo contenido puede leer el usuario a través de visores hexadecimales. B1

9.2. Dibujar el esquema lógico a nivel de bloques de la unidad de proceso general junto con la interconexión del subsistema de entrada/salida ($UPG+IO_{sim}$). B1

9.3. Indicar el valor de cada uno de los bits de la palabra de control que genera la unidad de control específica para que la unidad de proceso general junto con el subsistema de entrada/salida ($UPG+IO_{sim}$) realice en un ciclo una de las acciones del tipo `IN Rd, AddrPort`, `OUT AddrPort, Rb` o cualquiera de las otras acciones que se pueden realizar en la UPG. B2

9.4. Completar el cronograma simplificado con el valor de algunas señales/buses de la $UPG+IO_{sim}$ a partir del grafo de estados de la unidad de control de propósito específico que la gobierna y del estado del procesador (estado en el que se encuentra la unidad de control y valor de los registros de la UPG y puertos de IO_{sim} en el primer ciclo en el que se debe completar el cronograma). [8] [10]. B2

Protocolo de comunicación asíncrona de cuatro pasos

9.5. Dibujar un cronograma con las señales involucradas en una comunicación asíncrona entre dos circuitos secuenciales que tienen distintas señales de reloj. Expresar con palabras propias cómo funciona el protocolo y por qué se efectúa correctamente la comunicación independientemente de las velocidades de los relojes de los procesadores. Considerar dos casos, cuando la comunicación la inicia el emisor y cuando la inicia el receptor. B1 B1

PPE con UCE+UPG+IOsim con ES asíncrona con los puertos de REQ, ACK, DATA

9.6. Completar el diseño de un procesador de propósito específico (PPE) formado por una unidad de control de propósito específico (UCE) y la unidad de proceso general (UPG) para que junto con el subsistema de entrada/salida sencillo (IO_{sim}) realice una funcionalidad concreta. La unidad de control sólo tiene una entrada, el bit z que sale de la ALU de la UPG. A partir de la descripción funcional del PPE y del grafo de estados incompleto de la unidad de control, completar el grafo para que implemente la funcionalidad descrita. Las entradas y salidas de datos son asíncronas y utilizan el protocolo de handshaking de cuatro fases sobre puertos que hacen las funciones de las señales Req y Ack. [7] [8] [9]. B2

9.7. Diseñar un procesador de propósito específico (PPE) formado por una unidad de control de propósito específico (UCE) y la unidad de proceso general (UPG) para que junto con el subsistema de entrada/salida sencillo (IO_{sim}) realice una funcionalidad concreta. La unidad de control sólo tiene una entrada, el bit z que sale de la ALU de la UPG. A partir de la descripción funcional del PPE y del grafo de estados incompleto de la unidad de control, completar el grafo para que implemente la funcionalidad descrita. Las entradas y salidas de datos son asíncronas y utilizan el protocolo de handshaking de cuatro fases sobre puertos que hacen las funciones de las señales Req y Ack. [8] [9]. A

Controladores con efecto lateral: teclado e impresora

9.8. Dibujar el esquema lógico interno del subsistema de entrada/salida ($IO_{KeyPrint}$) que consiste en un controlador sencillo de teclado y otro de impresora, con sus correspondientes puertos de datos y de estado (el acceso al registro de datos tiene un efecto lateral sobre el registro de estado: lo pone a cero). Dibujar el esquema interno del registro de estado. B1

9.9. Dibujar el esquema lógico a nivel de bloques del PPE formado por el procesador (UCE+UPG) conectado al subsistema de entrada/salida con teclado e impresora ($IO_{KeyPrint}$). Se dibujará el esquema lógico interno del procesador a nivel de bloques y su interconexión con el bloque $IO_{KeyPrint}$. B1

9.10. Explicar con palabras propias el protocolo de handshaking de cuatro pasos que se establece entre el controlador del teclado o de la impresora y el procesador mediante el método de "encuesta activa" (polling) sobre el registro de estado y el efecto lateral en la lectura (`IN`) o escritura (`OUT`) del registro de datos del controlador. B1

9.11. Completar el cronograma simplificado con el valor de algunas señales/buses del sistema UCE+UPG+ $IO_{KeyPrint}$ durante los ciclos que se realiza una funcionalidad descrita por un fragmento del grafo de estados de la UCE (especificadas las salidas con mnemotécnicos) y del estado inicial del sistema (estado de los registros del PPE y de los puertos de entrada salida). [10]. B2

9.12. Completar el grafo de estados incompleto, de como máximo 10 estados, de la UCE para que el sistema formado por UCE+UPG+$IO_{KeyPrint}$ realice la funcionalidad descrita mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel. La funcionalidad incluye la entrada/salida de datos por el teclado/impresora del $IO_{KeyPrint}$. El grafo original es incompleto porque le falta algún nodo y/o arco y/o sus etiquetas como por ejemplo el mnemotécnico de la palabra de control de algunos nodos. B2

9.13. Dibujar el grafo de estados, de como máximo 10 estados, de la UCE para que el sistema formado por UCE+UPG+$IO_{KeyPrint}$ realice la funcionalidad descrita mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel. La funcionalidad incluye la entrada/salida de datos por el teclado/impresora. El grafo original es incompleto porque le falta algún nodo y/o arco y/o sus etiquetas como por ejemplo el mnemotécnico de la palabra de control de algunos nodos. A

10. Unidad de control general

Comprender las similitudes y diferencias entre una unidad de control de propósito específico y una de propósito general y saber justificar la conveniencia del secuenciamiento implícito y de la codificación de las instrucciones mediante un formato compacto.

Conocer el formato, codificación y semántica de las instrucciones del lenguaje máquina SISA y su especificación en lenguaje ensamblador (excepto para las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR` que se verán más adelante). Escribir pequeños programas en este lenguaje.

Comprender la estructura interna y el funcionamiento del sistema UCG+UPG+$IO_{KeyPrint}$ formado por la Unidad de Control General que controla al sistema UPG+$IO_{KeyPrint}$, en el que cada instrucción se ejecuta un ciclo.

De la Unidad de control específica (UCE) a la de la Unidad de control general (UCG)

10.1. Razonar las ventajas e inconvenientes del secuenciamiento implícito frente al explícito y del implícito absoluto frente al implícito relativo al PC. B1

10.2. Razonar las ventajas e inconvenientes de la codificación de las instrucciones con un formato compacto frente a que cada bit de la palabra de control tenga una funcionalidad única. B1

Justificar la creación de cada una de las instrucciones del lenguaje SISA (excepto para las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`).

10.3. Dibujar el esquema lógico, a nivel de bloques, de la unidad de control general, UCG (formada por el registro contador de programa (PC), el secuenciador (que permite escribir el PC con PC+2 o con PC+2+SE(N8)*2), la memoria de instrucciones y la lógica de control). B1

10.4. Razonar sobre aspectos relacionados con la ejecución de las instrucciones almacenadas en la memoria de instrucciones de la unidad de control general, UCG, entre las que aparecerán instrucciones de ruptura de secuencia, a partir de las condiciones necesarias para que el resultado correcto del razonamiento sea único. B2

Formato, codificación y semántica de las instrucciones SISA

10.5. Ensamblar y desensamblar código SISA (excepto las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`) ayudados por una tabla que contiene el formato y codificación de las instrucciones. B2

10.6. Indicar cómo queda modificado el estado del computador UCG+UPG+$IO_{KeyPrint}$ después de la ejecución de una instrucción o al final de la ejecución de pequeños programas escritos en lenguaje ensamblador SISA (excepto las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`) de como máximo 10 instrucciones, a partir de un estado inicial del computador (en este sistema el estado del computador es el contenido de los registros del procesador y de los puertos de entrada y de salida). B2

10.7. Completar la escritura de un fragmento de código en lenguaje ensamblador SISA (excepto las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`), de como máximo 10 instrucciones, cuya funcionalidad se especifica mediante un texto, mediante un fragmento de código en un pseudolenguaje de alto nivel o mediante el grafo de estados de una unidad de control específica, para que el sistema UCG+UPG+$IO_{KeyPrint}$ implemente esa funcionalidad. El fragmento de código original es incompleto porque le falta alguna instrucción y/o parte de alguna instrucción (el mnemotécnico de la operación o de alguno de los operandos). B2

10.8. Escribir fragmentos de código en lenguaje ensamblador SISA (excepto las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`), de como máximo 10 instrucciones, cuya funcionalidad se especifica mediante un texto, mediante un fragmento de código en un pseudolenguaje de alto nivel o mediante el grafo de estados de una unidad de control específica, para que el sistema UCG+UPG+$IO_{KeyPrint}$ implemente esa funcionalidad. A

Modificación de la ALU para poder ejecutar la instrucción `MOVHI`

10.9. Dibujar el símbolo y el esquema lógico interno del bloque combinacional MOVHI(X,Y) con sólo cables, que tiene dos buses de entrada X e Y y uno de salida W de 16 bits. (Los 8 bits de menor peso de W son igual a los 8 de menor peso de X mientras que los 8 de más peso son igual a los 8 de menos peso de Y). B1

10.10. Dibujar el símbolo y el esquema lógico interno de la unidad aritmético lógica, ALU, usada para implementar la UPG del sistema UCG+UPG+$IO_{KeyPrint}$. La ALU es igual a la ALUupg definida en el objetivo 8.3 excepto que cuando OP_u = 2 y F = 010 en el que la salida W es igual a la salida del bloque MOVHI definido en el objetivo 10.9. B1

Palabra de Control

10.11. Escribir el valor de los bits de la Palabra de Control, con el máximo número de x (bits que pueden valer indistintamente 0 o 1) que podría generar una hipotética Lógica de Control para que se ejecutara correctamente una instrucción SISA dada (excepto las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`) en el sistema UCG+UPG+$IO_{KeyPrint}$ en el que la UPG se hubiera implementado con dicha Lógica de Control. B2

Lógica de Control (bloque CONTROL LOGIC)

10.12. Dibujar el esquema lógico interno de la Lógica de Control de la UCG (bloque CONTROL LOGIC). B1

10.13. Escribir el contenido de la tabla que especifica el contenido de la ROM del bloque CONTROL LOGIC a partir del esquema lógico del bloque CONTROL LOGIC y de la tabla resumen del formato y codificación de las instrucciones SISA. B2

10.14. Escribir el valor de los bits de la Palabra de Control que genera el bloque CONTROL LOGIC para ejecutar una instrucción SISA (excepto para las instrucciones `LD`, `ST`, `LDB`, `STB` y `JALR`) en el sistema UCG+UPG+$IO_{KeyPrint}$. Especificad como x los valores de los bits de la palabra de control que no se pueden conocer por no saber cómo se han implementado las x de la tabla del contenido de la ROM del bloque CONTROL LOGIC. B2

11. Memoria

Dibujar el esquema y comprender el funcionamiento del subsistema de memoria de datos (Mem) de 64KB formada por dos módulos de memoria entrelazada con opciones de lectura y escritura de byte y de word (16 bits).

Conocer el formato, codificación y semántica de las instrucciones `LD`, *load*, `ST`, *store*, `LDB`, *load byte*, y `STB`, *store byte*, del lenguaje máquina SISA y su especificación en lenguaje ensamblador. Escribir pequeños programas en lenguaje SISA (excepto con la instrucción `JALR` que se verá más adelante).

Comprender la estructura interna y el funcionamiento del sistema UCG+UPG+$IO_{KeyPrint}$+MEM formado por el sistema UCG+UPG+$IO_{KeyPrint}$ junto con el subsistema de memoria de datos Mem, en el que cada instrucción se ejecuta en un ciclo.

11.1. Dibujar el símbolo de una memoria RAM indicando sus entradas y salidas. Explicar con palabras propias la funcionalidad de una memoria RAM vista desde sus entradas y salidas y considerando los tiempos característicos de una lectura y de una escritura. Dibujar el cronograma genérico de la lectura o la escritura de una memoria RAM, indicando los tiempos característicos. B1

11.2. Dibujar el esquema lógico del subsistema de memoria de datos (Mem) de 64KB formada por dos módulos de memoria entrelazada con opciones de lectura y escritura de byte y de word (16 bits). B1

11.3. Explicar con palabras propias qué hacen las instrucciones `LD`, *load*, `ST`, *store*, `LDB`, *load byte*, y `STB`, *store byte*, del lenguaje máquina SISA cuando se ejecutan en el sistema UCG+UPG+$IO_{KeyPrint}$+MEM y justificar el formato que utilizan para su codificación en lenguaje máquina SISA. Expresar formalmente, en un lenguaje de alto nivel, la semántica de estas nuevas instrucciones. B1

11.4. Ensamblar y desensamblar código SISA (excepto al instrucción `JALR`) con instrucciones `LD` (*load*), `ST` (*store*), `LDB` (*load byte*) y `STB` (*store byte*). B2

11.5. Dibujar el esquema lógico a nivel de bloques del sistema UCG+UPG+$IO_{KeyPrint}$+MEM. B1

11.6. Indicar cómo queda modificado el estado del computador UCG+UPG+$IO_{KeyPrint}$+MEM después de la ejecución de una instrucción o al final de la ejecución de pequeños programas escritos en lenguaje ensamblador SISA (excepto la instrucciones `JALR`) de como máximo 10 instrucciones, a partir de un estado inicial del computador (en este sistema el estado del computador es el contenido de los registros del procesador, de la memoria y de los puertos de entrada y de salida). B2

11.7. Dibujar el esquema lógico interno de la nueva Lógica de Control de la UCG (bloque CONTROL LOGIC) para que pueda generar las nuevas señales Byte y Wr-Mem de la palabra de control. B1

11.8. Escribir el contenido de la tabla que especifica el contenido de la ROM del bloque CONTROL LOGIC del sistema UCG+UPG+$IO_{KeyPrint}$+MEM a partir del esquema lógico del bloque CONTROL LOGIC y de la tabla resumen del formato y codificación de las instrucciones SISA (excepto para la instrucción JALR). B2

11.9. Escribir el valor de los bits de la Palabra de Control que genera el bloque CONTROL LOGIC para ejecutar una instrucción SISA (excepto para la instrucción JALR) en el sistema UCG+UPG+$IO_{KeyPrint}$+MEM. Especificad como x los valores de los bits de la palabra de control que no se pueden conocer por no saber cómo se han implementado las x de la tabla del contenido de la ROM del bloque CONTROL LOGIC. B2

11.10. Completar el cronograma simplificado con el valor de algunas señales/buses del sistema UCG+UPG+$IO_{KeyPrint}$+MEM durante la ejecución de un fragmento de código SISA (excepto con la instrucción JALR), que se dará en ensamblador, y del estado inicial del computador. [10] B2

11.11. Completar un código ensamblador SISA (excepto con la instrucción JALR) para que cuando se ejecute en el sistema UCG+UPG+$IO_{KeyPrint}$+MEM realice una funcionalidad descrita mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel que incluya acceso a la memoria de datos. B2

11.12. Escribir un código ensamblador SISA (excepto con la instrucción JALR) para que cuando se ejecute en el sistema UCG+UPG+$IO_{KeyPrint}$+MEM realice una funcionalidad descrita mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel que incluya acceso a la memoria de datos. A

12. Computadores SISC Harvard uniciclo y multiciclo

(El computador SISC Harvard uniciclo es el mismo sistema UCG+UPG+$IO_{KeyPrint}$+MEM, que se diseñó en el tema anterior, pero con la generación de las señales de permiso de modificación del estado del computador correctamente generadas teniendo en cuenta las restricciones temporales del hardware usado).

Comprender la estructura interna del computador SISC Harvard uniciclo, en el que cada instrucción tarda un ciclo en ejecutarse, y su funcionamiento al ejecutar código SISA.

Comprender la estructura interna del computador SISC Harvard multiciclo, en el que hay instrucciones que se ejecutan en tres ciclos y otras en cuatro, y su funcionamiento al ejecutar código SISA.

SISC Harvard uniciclo

12.1. Completar el diseño de la lógica de control del sistema UCG+UPG+$IO_{KeyPrint}$+MEM, tal como quedo en el tema anterior, para que genere correctamente las señales de permiso de modificación del estado del computador (WrD, Wr-Out, Rd-In, Wr-Mem) de acuerdo con las restricciones temporales del hardware usado, que pueden ser diferentes a los del sistema original) y de los tiempos a 1, T1, y a 0, T0, de un periodo (ciclo) de la señal de reloj (siendo el tiempo de ciclo Tc = T1+T0). B1

12.2. Dibujar el esquema a bloques del computador SISC Harvard uniciclo y el esquema lógico interno de cada bloque mediante la interconexión de bloques más pequeños, hasta el nivel de puertas Not, And y Or, memorias ROM (indicando su contenido), módulos de memoria RAM con unidad de direccionamiento de byte y biestables D. B1

12.3. Indicar el contenido de la ROM de la lógica de control (ROM_CTRL_LOGIC) a partir del esquema lógico del bloque SISC Harvard unicycle CONTROL LOGIC y de la tabla resumen del formato y codificación de las instrucciones SISA (excepto para la instrucción JALR). B2

12.4. Indicar el valor de algunas señales o buses de la Unidad de Proceso y de la Unidad de Control del computador SISC Harvard uniciclo, una vez estabilizadas, en el ciclo en que se B2

ejecuta una instrucción concreta o en cada ciclo de la ejecución de una secuencia de unas pocas instrucciones SISA (cronograma simplificado y/o modificación del estado del computador al ejecutar una secuencia de instrucciones).

12.5. Especificar el camino crítico (o uno de ellos si hay varios) y calcular el tiempo de ciclo mínimo del computador SISC Harvard uniciclo (o de una pequeña variante de este computador), dados los tiempos de propagación de los elementos que lo forman (que pueden ser diferentes a los originales). También se puede pedir lo anterior pero suponiendo que sólo existe un tipo de instrucción (camino crítico de la ejecución de un tipo de instrucción concreto). B2

12.6. Completar el diseño del computador SISC Harvard uniciclo parcialmente modificado para que cumpla nuevas restricciones respecto del hardware disponible y/o del tiempo de ciclo. B2

12.7. Proponer modificaciones del computador SISC Harvard uniciclo para que cumpla nuevas restricciones respecto del hardware disponible y/o del tiempo de ciclo. A

12.8. Calcular el tiempo de ejecución, en el computador SISC Harvard uniciclo, de un programa que se da escrito en ensamblador SISA. B2

SISC Harvard multiciclo

12.9. Justificar las ventajas/desventajas una implementación multiciclo v.s. una uniciclo de dos computadores que ejecutan el mismo lenguaje máquina (como es el caso de los SISC Harvard uni y multiciclo que ejecutan el lenguaje SISA). Responder a preguntas de solución única sobre estas justificaciones. B1

12.10. Justificar las modificaciones necesarias para transformar el SISC Harvard uniciclo en el SISC Harvard multiciclo. Responder a preguntas de solución única sobre estas justificaciones. B2

12.11. Dibujar la unidad de control (formada por el PC, el secuenciador, la memoria de instrucciones y el bloque SISC Harvard Multicycle CONTROL) del computador SISC Harvard multiciclo a nivel de bloques y saber dibujar el esquema lógico interno de cada bloque mediante la interconexión de bloques más pequeños, hasta el nivel de puertas Not, And, Or, memorias ROM (indicando su contenido), módulos de memoria RAM con unidad de direccionamiento de byte y biestables D. Dibujar el grafo de estados del circuito secuencial que se encuentra implementado en el interior del bloque SISC Harvard Multicycle CONTROL. B1

12.12. Especificar el camino crítico (o uno de ellos si hay varios) y calcular el tiempo de ciclo mínimo del computador SISC Harvard multiciclo (o de una pequeña variante de este computador), dados los tiempos de propagación de los elementos que lo forman (que pueden ser diferentes a los originales). También se puede pedir lo anterior pero suponiendo que sólo existe un tipo de instrucción (camino crítico de la ejecución de un tipo de instrucción concreto). B2

Evaluación del rendimiento del SISC Harvard multiciclo frente al uniciclo

12.13. Calcular el tiempo de ejecución, en el computador SISC Harvard multiciclo, de un programa que se da escrito en ensamblador SISA y compararlo con el tiempo de ejecución en el computador SISC Harvard uniciclo. Calcular el porcentaje de aumento/disminución en la velocidad de ejecución, o en el tiempo de ejecución, entre los dos computadores SISC Harvard (el uniciclo y el multiciclo), para el mismo programa. B2

13. Computador SISC

13.1. Explicar con palabras propias y/o responder a preguntas de solución única sobre las ventajas/desventajas (respecto de la velocidad de ejecución de los programas, cantidad de Hardware usado y versatilidad) de cada uno de los tres computadores que pueden ejecutar código SISA: SISC Harvard uniciclo, SISC Harvard umulticiclo y SISC von Neumann. B1

13.2. Dibujar el esquema a bloques del computador SISC von Neumann y el esquema lógico interno de cada bloque mediante la interconexión de bloques más pequeños, hasta el nivel de puertas Not, And y Or, memorias ROM (indicando su contenido), módulos de memoria RAM con unidad de direccionamiento de byte y biestables D. B1

13.3. Expresar el contenido de las dos ROMs del bloque SISC CONTROL UNIT (ROM_Q+ y ROM_OUT) a partir del esquema lógico interno de este bloque y de la tabla resumen del formato y codificación de las instrucciones SISA. B1

13.4. Indicar el valor de algunas señales o buses de la Unidad de Proceso y de la Unidad de Control del computador SISC von Neumann, una vez estabilizadas, para un ciclo concreto o para una secuencia de ciclos (cronograma simplificado) durante la ejecución de una o varias (dos o tres) instrucciones SISA. Se debe poner x como valor de un bit siempre que no se pueda saber el valor de ese bit (por no saber cómo se han implementado las x en el contenido de las dos ROMs de la unidad de control o, en general, porque en el enunciado no se ha dado la suficiente información para saber el valor de ese bit). B2

13.5. Escribir el valor de los bits de la Palabra de Control que genera el bloque SISC CONTROL UNIT durante un ciclo concreto de la ejecución de una instrucción SISA (se dará el nodo del grafo de estados en el que se encuentra la unidad de control en ese ciclo) o durante una secuencia de ciclos que puede incluir la ejecución de varias (dos o tres) instrucciones (se dará el nodo del primer ciclo de la secuencia). Especificad como x los valores de los bits de la palabra de control que no se pueden conocer por no saber cómo se han implementado las x de la tabla del contenido de la ROM_OUT de la unidad de control del SISC von Neumann. (Este objetivo es un caso particular del 13.4) B2

13.6. Indicar cómo queda modificado el estado del computador SISC von Neumann después de la ejecución de una instrucción o al final de la ejecución de pequeños programas escritos en lenguaje ensamblador SISA de como máximo 10 instrucciones, a partir de un estado inicial del computador. B2

13.7. A partir del computador SISC von Neumann parcialmente modificado para que pueda ejecutar una nueva instrucción de lenguaje máquina completar el diseño para que el nuevo computador ejecute correctamente las instrucciones originales más la nueva instrucción. Se parte de la definición de la semántica de la nueva instrucción y de su codificación y/o de la modificación (completa o incompleta) de la unidad de proceso y/o de la unidad de control. B2

13.8. Rediseñar el computador SISC Harvard uniciclo para que pueda ejecutar una nueva instrucción de lenguaje máquina además de las instrucciones originales a partir de la semántica de la nueva instrucción. A

- Proponer una codificación adecuada para la nueva instrucción y justificar la elección.
- Dibujar el esquema lógico de la unidad de proceso propuesta, o los cambios producidos sobre la unidad de proceso para crear la nueva instrucción, justificando la propuesta.
- Dibujar el esquema lógico de la unidad de control propuesta, o los cambios producidos sobre la unidad de control original para crear la nueva unidad de control, justificando la propuesta. Además, indicar los contenidos (o los cambios en los contenidos) de las dos ROMs de la unidad de control y dibujar el nuego grafo de estados de la unidad de control (o los cambios sobre el original).

13.9. Completar el diseño del computador SISC von Neumann parcialmente modificado para que cumpla restricciones respecto del hardware disponible y/o del tiempo de ciclo. B2

13.10. Rediseñar el computador SISC von Neumann para que cumpla restricciones respecto del hardware disponible y/o del tiempo de ciclo. Dibujar el esquema lógico de las partes que se A

modifiquen, los contenidos de las dos ROMs de la unidad de control y el nuevo grafo de estados. Justificar los cambios propuestos.

13.11. Especificar el camino crítico (o uno de ellos si hay varios) y calcular el tiempo de ciclo mínimo del computador SISC von Neumann (o de una pequeña variante de este computador), dados los tiempos de propagación de los elementos que lo forman (que pueden ser diferentes a los originales). También se puede pedir lo anterior pero suponiendo que sólo existe un tipo de instrucción (camino crítico de la ejecución de un tipo de instrucción concreto y tiempo de ciclo mínimo si solo existiera ese tipo de instrucción) o para un ciclo concreto de la ejecución de una instrucción concreta (camino crítico de un ciclo/nodo de una de las fases de ejecución de una instrucción y tiempo de ciclo mínimo si este es el nodo que más tiempo requiere). B2

14. Lenguaje máquina y ensamblador SISA

Conocer el juego de instrucciones del lenguaje máquina SISA y el juego de instrucciones y directivas del lenguaje ensamblador SISA. Ensamblar código SISA.

Escribir pequeños programas en lenguaje ensamblador SISA.

14.1. Describir con palabras propias qué son, para qué se utilizan y qué hacen las etiquetas, las funciones `hi` y `lo` y las directivas del ensamblador del lenguaje SISA. B1

14.2. A partir de un fragmento de programa en ensamblador SISA en el que se definen los datos con etiquetas y directivas (en la sección `.data`) y las instrucciones (en la sección `.text`), ensamblar el programa: escribir el contenido (en hexadecimal) que debe tener la memoria para que el programa se ejecute correctamente una vez cargado, dadas las direcciones de memoria donde se carga la sección `.data` y la `.text`. B2

14.3. Indicar cómo queda modificado el estado del computador (contenido de registros, memoria y puertos de entrada y de salida) después de la ejecución de una instrucción o al final de la ejecución de pequeños programas escritos en lenguaje ensamblador SISA (como máximo 15 instrucciones incluyendo directivas), a partir de un estado inicial del computador. B2

14.4. Completar la escritura pequeños programas en lenguaje ensamblador SISA o en SISA extendido con alguna nueva instrucción de complejidad equivalente a las originales (de como máximo 15 instrucciones incluyendo directivas) cuya funcionalidad se especifica mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel. El programa original es incompleto porque le falta alguna instrucción y/o parte de alguna instrucción (el mnemotécnico de la operación o de alguno de los operandos). B2

14.5. Escribir pequeños programas en lenguaje ensamblador SISA o en SISA extendido con alguna nueva instrucción de complejidad equivalente a las originales, (de como máximo 15 instrucciones incluidas directivas), cuya funcionalidad se especifica mediante un texto o mediante un fragmento de código en un pseudolenguaje de alto nivel. A